A

Project Report

on

Disaster Management Using Real-Time Web Data, Chatbot Assistance, and VR Simulation

submitted as partial fulfillment for the award of

BACHELOR OF TECHNOLOGY DEGREE

SESSION 2025-26

in

Computer Science & Engineering (AI & ML)

By

Kartik Chaudhary (2200291530053)
Avanish Kumar (2200291530030)
Ashutosh Mishra (2200291530028)
Animesh Kumar Singh (2200291530013)

Under the supervision of

Ms. Umang Kant

KIET Group of Institutions, Ghaziabad

Affiliated to

Dr. A.P.J. Abdul Kalam Technical University, Lucknow

(Formerly UPTU)

May, 2026

DECLARATION

We hereby declare that this submission is our own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

Name	Roll No.	Signature
Kartik	2200291530053	
Avanish Kumar	2200291530030	
Animesh Kumar Singh	2200291530013	
Ashutosh Mishra	2200291530028	

CERTIFICATE

This is to certify that Project Report entitled "Disaster Management Using Real-Time Web Data, Chatbot Assistance, and VR Simulation" which is submitted by "Kartik, Avanish Kumar, Animesh Kumar Singh, Ashutosh Mishra" in partial fulfillment of the requirement for the award of degree B. Tech. in Department of Computer Science & Engineering (AI&ML) of Dr. A.P.J. Abdul Kalam Technical University, Lucknow is a record of the candidates own work carried out by them under my supervision. The matter embodied in this report is original and has not been submitted for the award of any other degree.

Ms. Umang Kant
(Project Supervisor)

Dr. Rekha Kashyap
(Dean, CSE AIML)

Date:

ACKNOWLEDGEMENT

It gives us a great sense of pleasure to present the report of the B. Tech Project undertaken during B. Tech. Final Year. We owe special debt of gratitude to Ms. Umang Kant, Assistant Professor, Department of Computer Science & Engineering (AI&ML), KIET Group of Institutions, Ghaziabad, for her constant support and guidance throughout the course of our work. Her sincerity, thoroughness and perseverance have been a constant source of inspiration for us. It is only her cognizant efforts that our endeavors have seen light of the day.

We also take the opportunity to acknowledge the contribution of Dr. Rekha Kashyap, Dean, Computer Science & Engineering (AI&ML) Department, KIET Group of Institutions, Ghaziabad, for her full support and assistance during the development of the project. We also do not like to miss the opportunity to acknowledge the contribution of all the faculty members of the department for their kind assistance and cooperation during the development of our project. Last but not the least, we acknowledge our friends for their contribution in the completion of the project.

Name	Roll No.	Signature
Kartik	2200291530053	
Avanish Kumar	2200291530030	
Animesh Kumar Singh	2200291530013	
Ashutosh Mishra	2200291530028	

ABSTRACT

The growing frequency and impact of natural disasters highlight the need for fast and reliable access to accurate information. This paper presents a disaster management system that integrates real-time web data extraction, machine learning, chatbot assistance, and virtual reality (VR) simulation to enhance preparedness and response. The system collects disaster-related data from online sources using web scraping and processes it through natural language processing (NLP) techniques. Classification models such as Support Vector Machine (SVM) and Naive Bayes are used to filter and categorize information, helping identify critical alerts and reduce misinformation.

A Flutter-based mobile application delivers real-time alerts, supports offline access, and ensures usability in low-connectivity environments. An AI-powered chatbot provides instant guidance using a hybrid approach combining rule-based logic and language models. Additionally, a VR module developed with Unity offers immersive training experiences to improve disaster awareness and response skills.

By combining these components into a unified platform, the system addresses challenges such as information delays, lack of preparedness, and unreliable communication. The results demonstrate efficient performance and highlight the system's potential to support communities and improve disaster response outcomes.

TABLE OF CONTENTS

Page No.

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Full Form / Description
AI	Artificial Intelligence
API	Application Programming Interface
C#	C Sharp (Programming Language)
FCM	Firebase Cloud Messaging
FSM	Finite State Machine
GPS	Global Positioning System
LLM	Large Language Model
LSTM	Long Short-Term Memory
ML	Machine Learning
NLP	Natural Language Processing
REST	Representational State Transfer
SDK	Software Development Kit

CHAPTER 1

INTRODUCTION

1.1 PROJECT DESCRIPTION

In recent decades, the frequency and intensity of natural and man-made disasters have increased significantly, posing serious threats to human life, infrastructure, and economic stability. According to global reports, thousands of disasters have occurred in the past two decades, affecting billions of people worldwide and resulting in massive loss of life and property . Despite advancements in prediction systems and emergency response mechanisms, one of the biggest challenges remains the lack of timely, accurate, and verified information during disasters.

Traditional disaster management systems primarily rely on centralized communication channels, which often suffer from delays, limited accessibility, and lack of real-time updates. At the same time, social media and online platforms have emerged as major sources of real-time information. However, these platforms also introduce challenges such as misinformation, unverified reports, and data overload, making it difficult for authorities and citizens to identify reliable information quickly [8][9].

To address these issues, modern disaster management approaches are shifting towards technology-driven, data-centric solutions that integrate real-time data processing, machine learning, and user-centric applications. Techniques such as Natural Language Processing (NLP), Artificial Intelligence (AI), and cloud-based systems are increasingly being used to extract, classify, and distribute disaster-related information efficiently [7].

1.2 Motivation

The motivation behind this project arises from the critical gaps observed in existing disaster management systems:

- **Lack of a unified platform** to collect, verify, and distribute disaster-related information
- **Delayed response times** due to manual data processing and fragmented communication
- **Spread of misinformation** during emergencies, leading to panic and confusion
- **Limited public awareness and preparedness**, especially in rural or resource-constrained areas

Additionally, most existing systems focus only on information dissemination, ignoring the importance of user interaction and training. There is a need for systems that not only provide real-time updates but also educate users about disaster response through interactive and immersive technologies such as Virtual Reality (VR) [6].

This project aims to bridge these gaps by developing an integrated disaster management system that combines real-time data collection, intelligent classification, chatbot assistance, and VR-based training into a single platform.extract, classify, and distribute disaster-related information efficiently [7].

1.3 Objectives of the Project

The primary objectives of this project are:

- To develop a system for **real-time collection of disaster-related data** from web and social media sources
- To implement **machine learning-based classification** using techniques such as Support Vector Machine (SVM) and Naive Bayes for filtering and categorizing information
- To design an **AI-powered chatbot** that provides guidance before, during, and after disasters

- To create a **mobile application** that delivers alerts and supports offline access to critical information
- To develop a **VR-based simulation module** for disaster awareness and training
- To ensure the system is **scalable, reliable, and accessible** to a wide range of users

1.4 Scope of the Project

The scope of this project focuses on building a multi-modal disaster management platform that enhances preparedness and response through technology. The system is designed to:

- Collect and process data from **online sources in real time**
- Filter and classify information to identify **high-priority alerts**
- Deliver notifications and guidance through a **mobile application**
- Provide **interactive chatbot assistance** for user queries
- Offer **VR-based training simulations** for disaster preparedness

However, the system does not aim to replace existing government infrastructure or perform satellite-based forecasting or large-scale rescue coordination. Instead, it acts as a supportive tool that enhances awareness, improves communication, and assists both citizens and authorities during disaster situations.

CHAPTER 2

LITERATURE REVIEW

Disaster management has evolved significantly with the integration of advanced technologies such as Artificial Intelligence (AI), Machine Learning (ML), and real-time data analytics. Researchers have explored various approaches to improve disaster response, including social media analysis, automated classification systems, chatbot-based assistance, and immersive training through Virtual Reality (VR).

However, despite these advancements, most existing systems operate in isolation and fail to provide a comprehensive, integrated solution that addresses all phases of disaster management. This chapter reviews key contributions in the domain and highlights the gaps that motivate the proposed system.

2.1 Social Media and Web Data in Disaster Management

Social media platforms such as Twitter and Facebook have become critical sources of real-time information during disasters. Several studies highlight their importance in providing immediate updates, identifying affected areas, and coordinating rescue operations.

Research shows that disaster-related data collected from social media can be highly valuable but is often unstructured, noisy, and unreliable. The presence of fake news, duplicate information, and inconsistent language makes it difficult to extract meaningful insights [8][9].

To address this, researchers have used data mining and NLP techniques to filter and process large volumes of text data. However, the challenge of verifying authenticity and ensuring data credibility remains a significant limitation. Despite these advancements, existing navigation systems often lack integration with other assistive features such as object detection and text recognition. Additionally, accuracy issues in crowded or complex environments remain a challenge.

2.2 Machine Learning Techniques for Classification

Machine learning models such as Support Vector Machine (SVM) and Naive Bayes have been widely used for classifying disaster-related text into meaningful categories. These models are particularly effective for short-text classification tasks, such as tweets and alerts [7].

Studies indicate that SVM provides higher accuracy in multi-class classification problems, while Naive Bayes offers faster processing speed, making it suitable for real-time applications.

More recently, deep learning and transformer-based models have also been explored for improved accuracy. However, these models often require high computational resources, making them less suitable for lightweight, real-time systems.

Thus, a hybrid approach using traditional ML models remains practical for systems that prioritize speed, efficiency, and scalability.

2.3 Chatbot Systems in Disaster Response

Chatbots have emerged as an effective tool for communication during emergencies. They can provide instant responses, guide users with safety instructions, and reduce the burden on emergency services.

Early chatbot systems were primarily rule-based, limiting their ability to handle complex or dynamic queries. Recent advancements in Large Language Models (LLMs) have improved chatbot flexibility and contextual understanding [4].

Research suggests that combining rule-based systems with AI-driven models can create a hybrid chatbot architecture that balances reliability and adaptability. However, many existing implementations lack integration with real-time data sources, reducing their effectiveness during rapidly evolving disaster scenarios.

2.4 Virtual Reality in Disaster Training

Virtual Reality (VR) is increasingly being used as a training tool in disaster management. It allows users to experience simulated disaster scenarios in a controlled environment, improving awareness and preparedness.

Studies show that VR-based training enhances user engagement, decision-making skills, and retention of safety procedures [6]. It has been applied in scenarios such as earthquakes, floods, and fire emergencies. Despite its advantages, VR is often used as a standalone training tool and is rarely integrated with real-time disaster information systems. This limits its potential to provide a complete disaster management solution.

2.5 Limitations of Existing Systems

From the reviewed literature, several key limitations can be identified:

- Lack of **integration between data collection, processing, and user interaction systems**
- Difficulty in handling **misinformation and unverified data** from social media
- Limited use of **real-time automated classification** for prioritizing alerts
- Chatbot systems that lack **context awareness or real-time data integration**

Table 2.1: Comparison of Existing Approaches

Approach	Advantages	Limitations
Social Media Analysis	Real-time updates	Misinformation, noisy data
ML Classification	Automated categorization	Requires training data
Chatbots	Instant user interaction	Limited context (rule-based systems)
VR Training	High engagement & awareness	Not integrated with real-time data

CHAPTER 3

PROPOSED METHODOLOGY

This chapter describes the design and implementation of the proposed disaster management system. The system is developed as a multi-layered, modular platform that integrates real-time data acquisition, machine learning-based classification, intelligent chatbot interaction, mobile-based alert delivery, and Virtual Reality (VR) training.

The primary goal of the system is to collect, process, and disseminate disaster-related information efficiently while also improving user awareness and preparedness. The architecture ensures scalability, flexibility, and real-time responsiveness by connecting multiple independent components through REST APIs.

3.1 System Architecture Overview

The proposed system follows a service-based architecture consisting of five major components:

- **Data Acquisition Module (Python-based)**
- **NLP Classification Engine**
- **Backend Server (Node.js)**
- **Mobile Application (Flutter)**
- **VR Simulation Module (Unity)**

Each component operates independently but communicates through lightweight APIs, ensuring efficient data flow and scalability. The architecture is designed to handle real-time data streams while maintaining low latency and high reliability.

The workflow of the system can be summarized as follows:

1. Data is collected from web sources and social media
2. Relevant information is filtered and scored
3. Data is classified using machine learning models
4. Processed information is stored and served via backend APIs
5. Users receive alerts through the mobile app and chatbot

6. VR module provides disaster training simulations

Table 3.1: System Modules Description

Module	Technology Used	Function
Data Acquisition	Python, Selenium	Collect real-time data
NLP Classification	SVM, Naive Bayes	Categorize disaster information
Backend Server	Node.js	Manage APIs and data flow
Mobile Application	Flutter	Deliver alerts and user interaction
VR Simulation	Unity (C#)	Disaster training and awareness

3.2 Data Acquisition Pipeline

The data acquisition module is responsible for collecting real-time disaster-related information from various sources such as news websites and social media platforms.

- **Static content** is retrieved using Python libraries like Requests
- **Dynamic content** (JavaScript-rendered pages) is handled using Selenium
- A scheduled cron job ensures continuous data collection.

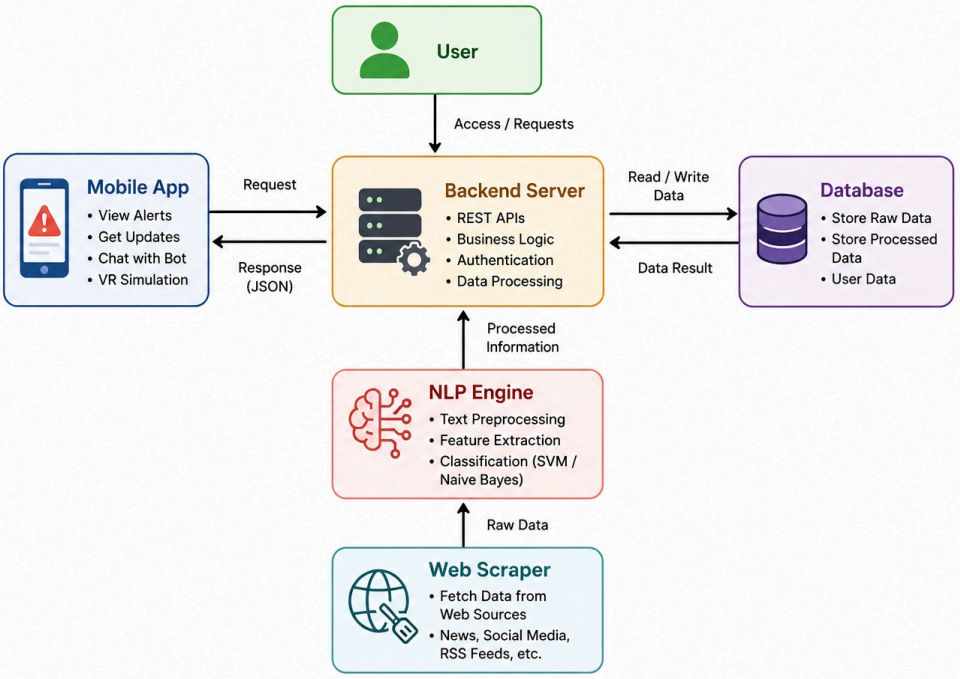


Figure 3.3: Data Flow Diagram (Level 0)

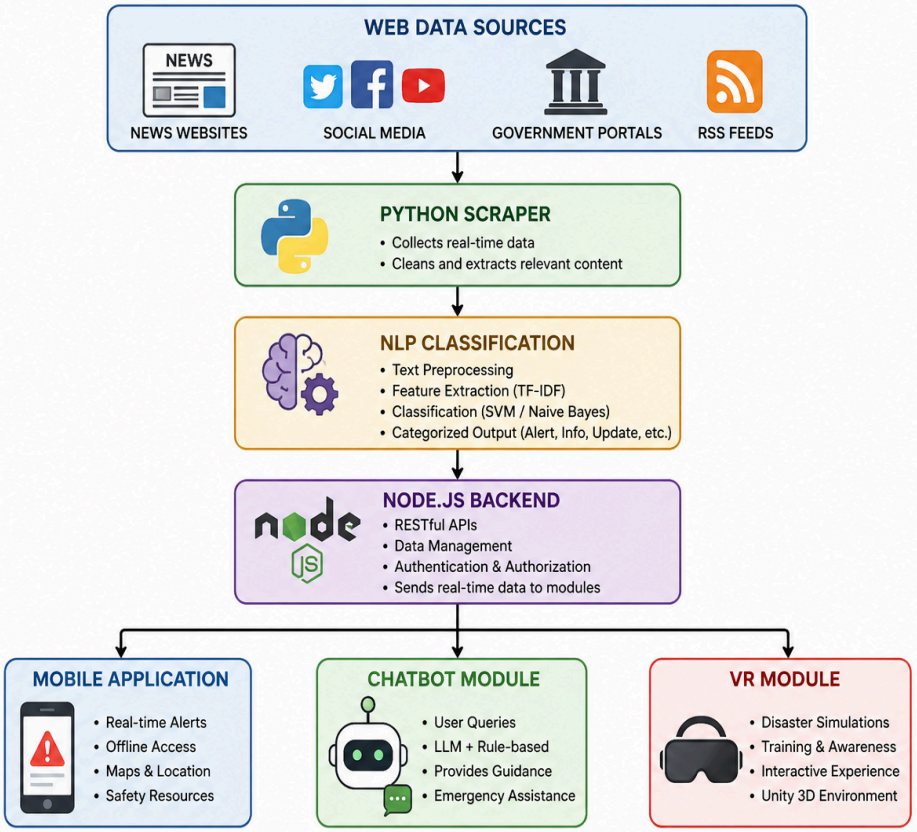


Figure 3.1: Proposed System Architecture

Each collected data item includes attributes such as:

- Text content
- Timestamp
- Source URL
- Associated media

To prioritize relevant information, a **relevance scoring mechanism** is applied:

$R=\alpha K+\beta T+\gamma S$

where:

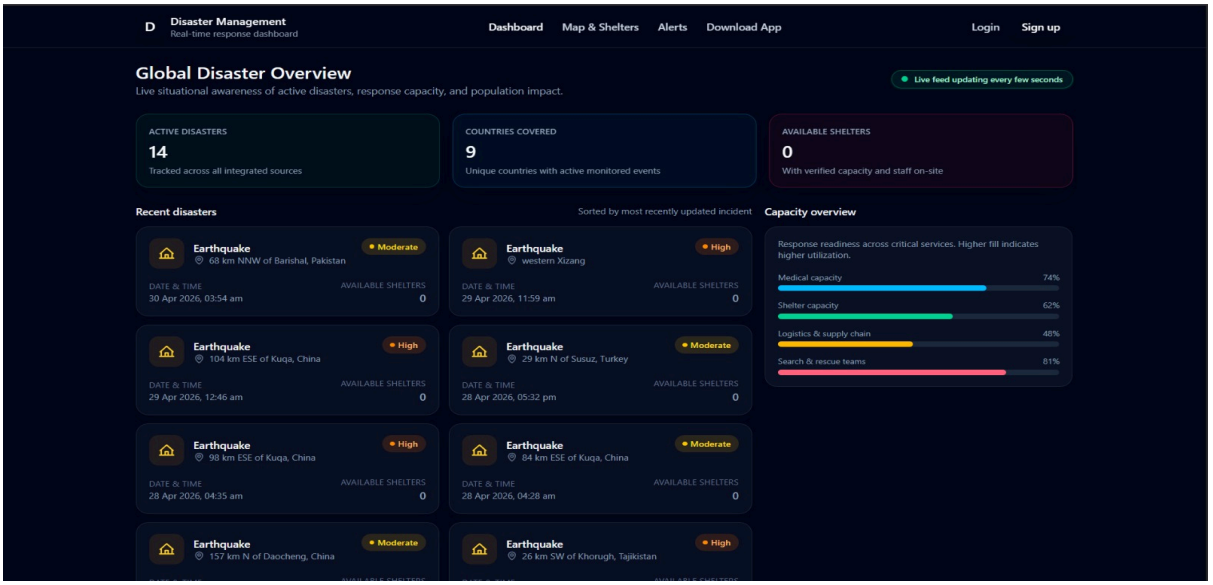
K represents keyword intensity

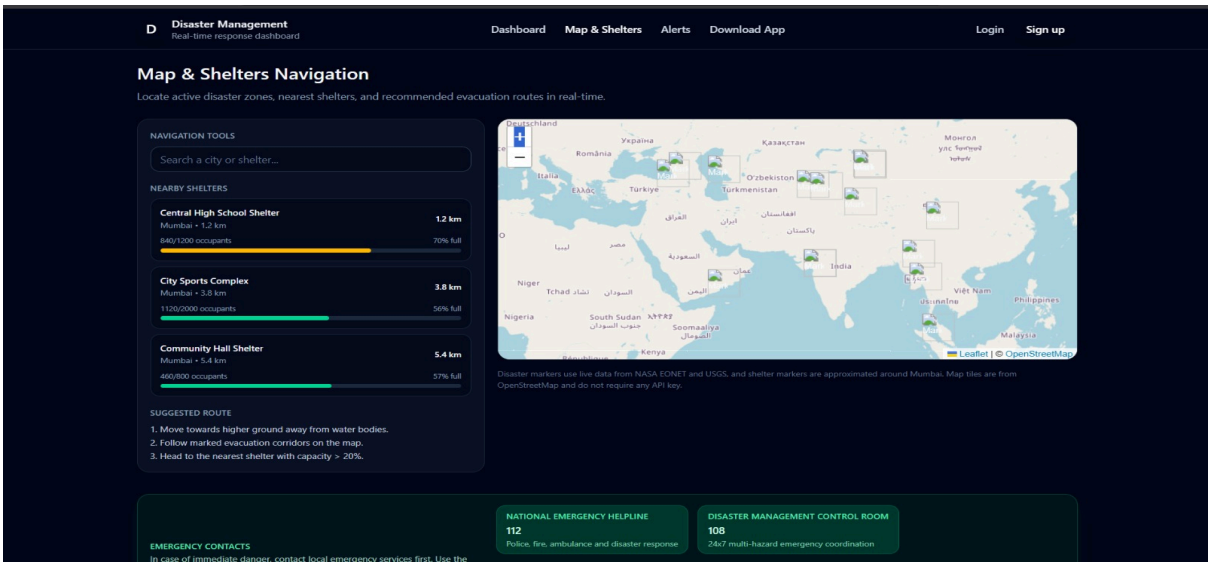
T represents time relevance

S represents source credibility

α,β,γ are weighted parameters

Only data exceeding a predefined threshold is forwarded to the classification module. This ensures that the system processes high-priority and meaningful information only, reducing noise and improving efficiency.





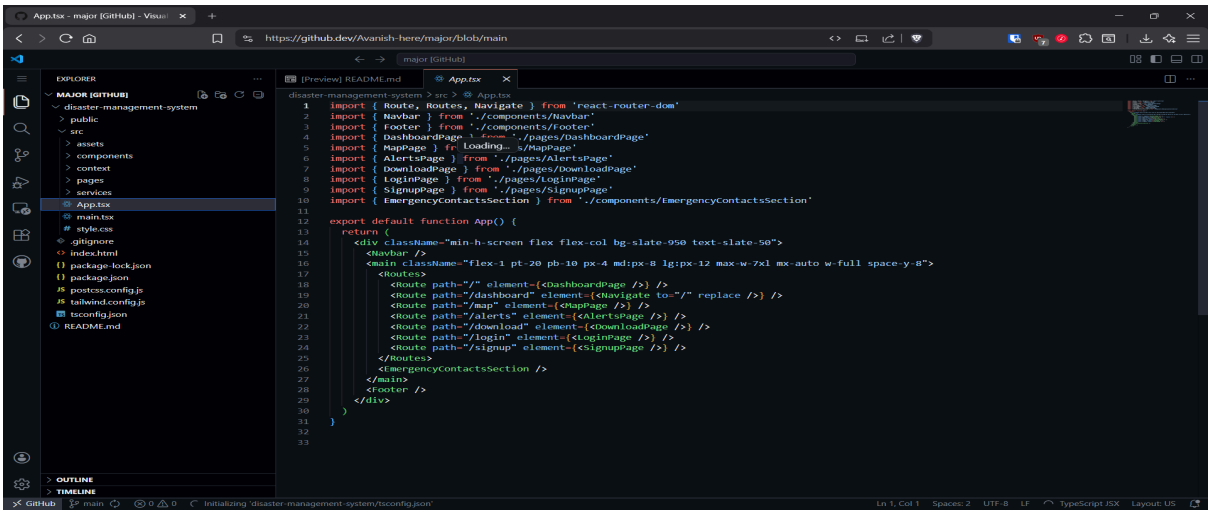


Figure 3.4: VS Code Development Environment for website

3.3 NLP-Based Classification Engine

The classification engine processes incoming text data using Natural Language Processing (NLP) techniques. The preprocessing steps include:

- Tokenization
- Lowercasing
- Stopword removal
- Stemming

The processed text is then converted into numerical form using TF-IDF vectorization.

$\hat{y} = \arg\max P(c \mid x)$

where:

x is the feature vector

c represents the set of possible classes

$\hat{y}$ is the predicted category

The system uses two classifiers:

- **Support Vector Machine (SVM):** High accuracy for multi-class classification
- **Multinomial Naive Bayes:** Faster inference for short text

The classification categories include:

- Alerts
- Government Notices
- Rescue Updates
- Public Advisories
- Casualty Reports

This hybrid approach ensures a balance between **accuracy and computational efficiency** [7][15].

3.4 AI Chatbot Architecture

The chatbot module is designed to provide interactive assistance to users during all phases of disaster management. It follows a **hybrid architecture** combining:

- Rule-based intent recognition
- AI-based response generation using language models

When a user query q is received, the system determines the most probable intent:

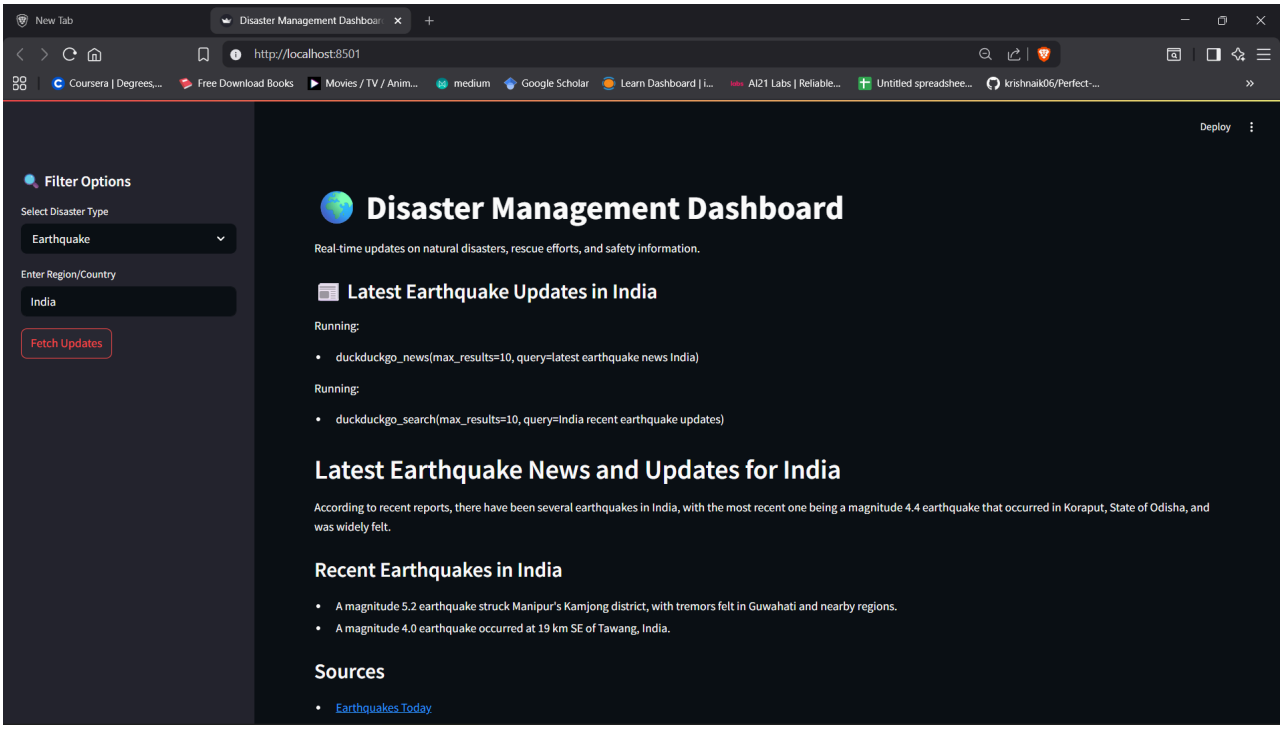
$I = arg\ max\ P(Ik \mid q)$

If the confidence level is high, the system responds using predefined rules. Otherwise, it forwards the query to a language model for dynamic response generation.

The chatbot supports:

- Pre-disaster guidance (preparedness tips)
- During-disaster assistance (emergency actions)
- Post-disaster support (recovery steps)

A simple **Finite State Machine (FSM)** is used to maintain conversation flow and handle multi-turn interactions. This design ensures both **reliability and flexibility** in user communication [4][12].



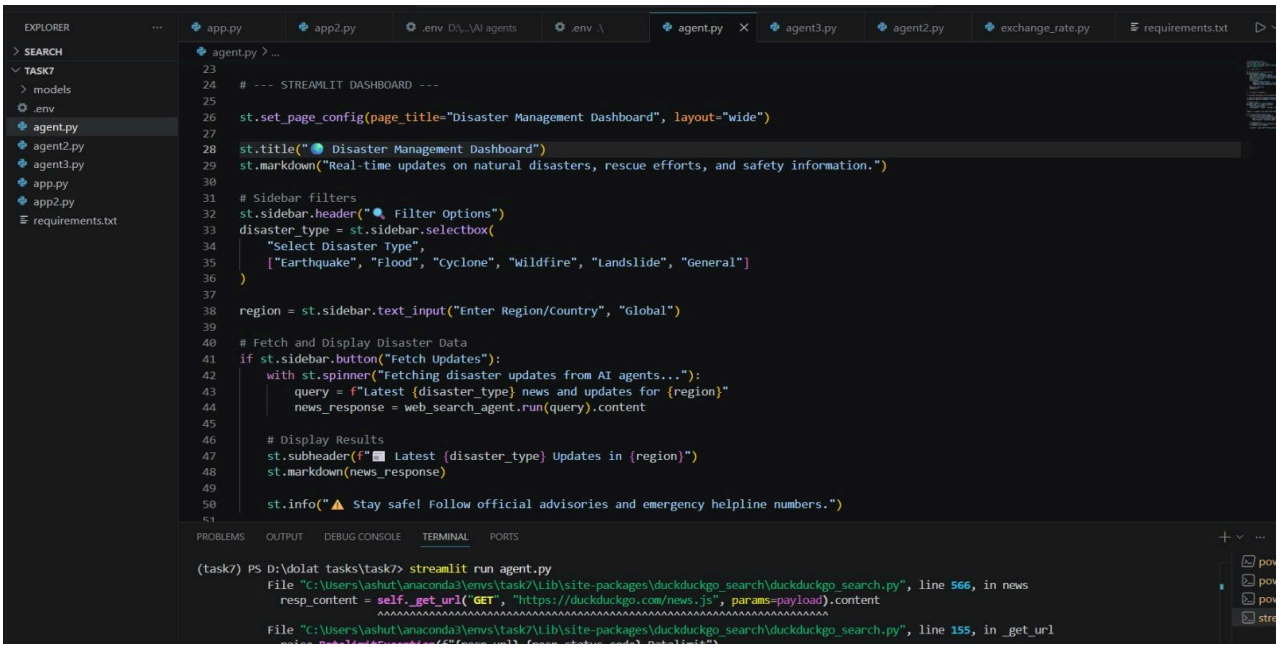


Figure 3.5: VS Code Development Environment for Web Scraping

3.5 Backend and API Integration

The backend is implemented using **Node.js**, which acts as a central communication layer between all modules. It provides RESTful APIs for:

- Data storage and retrieval
- Communication with the mobile application
- Integration with the chatbot
- Interaction with the VR module

A **MongoDB database** is used for storing structured and unstructured data. The backend is designed to handle concurrent requests efficiently and ensure real-time data availability.

3.7 Flutter Mobile Application

The mobile application serves as the primary user interface of the system. It is developed using Flutter to ensure cross-platform compatibility.

Key features include:

- **Real-time alerts** using Firebase Cloud Messaging (FCM)
- **Offline functionality** using SQLite database
- **Location-based notifications** using Google Maps API
- **Low-bandwidth mode** for better performance in poor network conditions

State management tools such as Bloc or Riverpod are used to maintain application stability and responsiveness. The app ensures that users can access critical information even without internet connectivity.

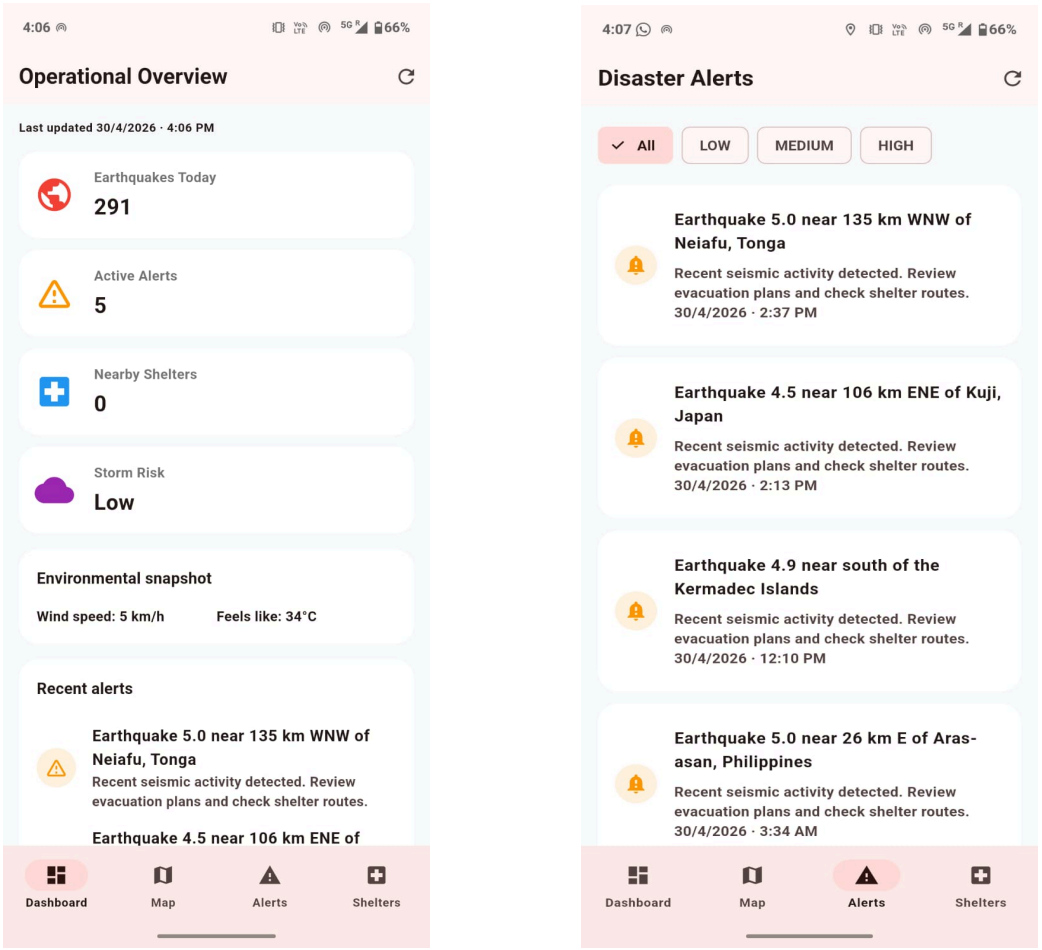


Figure 3.6: Mobile App Dashboard

3.8 VR Simulation Module

The VR module is developed using the Unity engine to provide immersive disaster training experiences. It simulates various disaster scenarios such as:

- Earthquakes
- Floods
- Fires

Key features include:

- Realistic physics using Rigidbody for earthquake simulation
- Fluid dynamics approximation for flood scenarios

- Interactive decision-making using C# scripts

Users can navigate different scenarios and learn appropriate safety measures through hands-on experience. This enhances **awareness, preparedness, and decision-making skills** during real-life disasters [6].

3.9 System Workflow Summary

The complete system operates through the following sequence:

1. Data is collected from web and social media sources
2. Relevance scoring filters important information
3. NLP models classify the data into predefined categories
4. Backend stores and manages processed data
5. Mobile application delivers alerts to users
6. Chatbot provides real-time assistance
7. VR module offers training simulations

This integrated workflow ensures that the system supports **both real-time response and long-term preparedness**.

CHAPTER 4

RESULTS AND DISCUSSION

This chapter presents the performance evaluation of the proposed disaster management system. The system is analyzed based on its key components, including data acquisition, classification accuracy, chatbot responsiveness, mobile application performance, and VR simulation efficiency.

The evaluation aims to measure how effectively the system processes real-time data, classifies information, and delivers timely alerts to users.

4.1 Evaluation Setup

The system was tested using disaster-related data collected from various online sources, including news websites and social media platforms, over a period of two weeks.

The evaluation focused on the following parameters:

- Data extraction latency
- Classification accuracy
- Response time of chatbot
- Mobile application performance
- VR simulation efficiency

These metrics were selected to ensure a comprehensive analysis of both **system performance and user experience**.

4.2 Performance Metrics

The performance of each module is summarized in Table 4.1.

Table 4.1: System Performance Evaluation

Component	Performance Metric
Data Scraping Pipeline	1.2 seconds average latency
Chatbot	0.5 seconds average response time
Mobile App	190 ms load time
VR Simulation	55–60 FPS
Naive Bayes Classifier	76% accuracy (faster inference)
SVM Classifier	82% accuracy

4.3 ANALYSIS OF RESULTS

4.3.1 Data Acquisition Performance

The data scraping module demonstrated efficient performance with an average latency of approximately **1.2 seconds per extraction cycle**. This indicates that the system can collect and process real-time data with minimal delay.

The use of optimized Python scripts and scheduling mechanisms ensured continuous and reliable data flow.

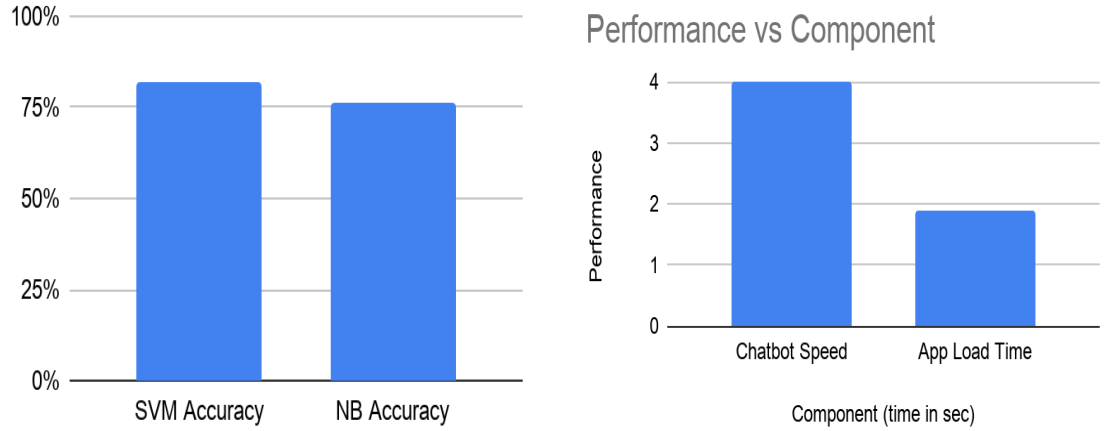


Figure 3.7: Bar Chart for Performance Metrics

4.3.2 Classification Accuracy

- 82% accuracy using SVM
- 76% accuracy using Naive Bayes

SVM performed better in handling multi-class classification problems, while Naive Bayes provided faster predictions for short text inputs.

This validates the effectiveness of using a **hybrid classification approach**, balancing both accuracy and speed [7].

4.3.3 Chatbot Performance

The chatbot responded to user queries with an average response time of **0.5 seconds**, demonstrating its ability to provide near real-time assistance.

The hybrid architecture (rule-based + AI model) allowed the chatbot to:

- Handle predefined queries efficiently
- Adapt to complex or unexpected inputs

This significantly improves user interaction during emergency situations [4].

4.3.4 Mobile Application Performance

The Flutter-based mobile application showed strong performance in both online and offline modes.

- Offline data loading time was approximately **190 milliseconds**, ensuring quick access to critical information
- Real-time alerts were delivered efficiently using push notification services

The inclusion of offline support makes the application highly useful in disaster scenarios where internet connectivity is limited.

4.3.5 VR Simulation Performance

The VR module maintained a frame rate of **55–60 FPS**, ensuring a smooth and immersive user experience.

The simulation successfully recreated disaster scenarios such as earthquakes and floods, allowing users to:

- Understand real-life situations
- Practice decision-making in a safe environment
- This enhances disaster preparedness and user engagement [6].

4.4 Discussion

The overall system demonstrates strong performance across all components. The integration of multiple technologies enables the system to:

- Deliver **real-time disaster updates**
- Accurately classify and filter information
- Provide **interactive user assistance** through chatbot
- Ensure accessibility through a **mobile application with offline support**
- Improve preparedness via **VR-based training**

One of the key strengths of the system is its **low latency and high responsiveness**, which are critical in disaster scenarios.

However, certain limitations were observed:

- Classification accuracy can be affected by noisy or ambiguous data
- Dependence on web sources may introduce variability in data quality
- VR module requires compatible hardware for full experience

Despite these limitations, the system performs effectively as a **comprehensive disaster management solution**.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

This project presented a comprehensive disaster management system that integrates real-time data processing, machine learning, chatbot assistance, mobile application support, and Virtual Reality (VR) training into a unified platform. The primary objective was to address the limitations of existing systems, particularly the lack of timely, reliable, and structured information during disaster situations.

The proposed system successfully demonstrates the ability to:

- Collect disaster-related data from multiple online sources in real time
- Filter and classify information using machine learning techniques such as SVM and Naive Bayes
- Provide interactive assistance through an AI-powered chatbot
- Deliver alerts and critical information via a mobile application with offline capability
- Enhance user preparedness through immersive VR-based simulations

The results obtained from system evaluation indicate that the platform performs efficiently with low latency, satisfactory classification accuracy, and fast response times. The integration of multiple modules into a single system improves both **information accessibility and user engagement**, making it a practical solution for modern disaster management scenarios .

Overall, the system contributes to improving disaster preparedness, reducing misinformation, and enabling faster response during emergencies. It highlights the importance of combining advanced technologies to create scalable and user-centric solutions that can assist both individuals and authorities.

5.2 Future Scope

Although the proposed system demonstrates strong performance, there are several opportunities for further enhancement and expansion:

- **Integration with Government APIs:**
  Direct integration with official disaster management authorities (such as NDMA) can improve the reliability and authenticity of alerts.
- **Predictive Analytics:**
  Incorporating advanced models such as LSTM can enable prediction of disaster trends, including flood levels, storm intensity, and aftershock probabilities [2].
- **Multilingual Support:**
  Extending the chatbot and mobile application to support multiple regional languages will increase accessibility, especially in rural areas.
- **Crowdsourced Reporting:**
  Allowing users to report incidents with credibility scoring can enhance real-time situational awareness and support faster response.
- **Cloud-Based Scalability:**
  Deploying the system on cloud platforms such as AWS or GCP can improve scalability, availability, and performance under high load conditions.
- **Advanced AI Models:**
  Integration of transformer-based models can further improve classification accuracy and contextual understanding.
- **Enhanced VR Experience:**
  Expanding VR scenarios with multi-user environments and more realistic simulations can improve training effectiveness.

These future enhancements can transform the system into a more **intelligent, scalable, and globally applicable disaster management platform**, capable of handling complex real-world scenarios more effectively.

References

[1] K. Sapountzaki, “Risk Mitigation, Vulnerability Management, and Resilience under Disasters,” *Sustainability*, vol. 14, no. 6, pp. 1–8, 2022.

[2] H. L. Tay, R. Banomyong, P. Varadejsatitwong, and P. Julagasigorn, “Mitigating Risks in the Disaster Management Cycle,” *Advances in Civil Engineering*, vol. 2022, Article ID 7454760, 2022.

[3] Anonymous, “Poverty and Disaster Resilience,” Independent Research Paper, 2020.

[4] A. Rossi, “The ERMES Chatbot: A Conversational Communication Tool for Improved Emergency Management and Disaster Risk Reduction,” *International Journal of Disaster Risk Reduction*, vol. 112, 2024.

[5] K. Kopanov, V. Varbanov, and T. Atanasova, “Leveraging Social Media Data and Artificial Intelligence for Improving Earthquake Response Efforts,” *arXiv preprint arXiv:2501.14767*, 2024.

[6] S. Khanal *et al.*, “Virtual and Augmented Reality in Disaster Management Technology: A Literature Review of the Past 11 Years,” *Frontiers in Virtual Reality*, vol. 3, 2022.

[7] F. Alam, F. Ofli, and M. Imran, “CrisisMMD: Multimodal Twitter Datasets from Natural Disasters,” *arXiv preprint arXiv:1805.00713*, 2018.

[8] T. H. Nazer *et al.*, “Intelligent Disaster Response via Social Media Analysis: A Survey,” *arXiv preprint arXiv:1709.02426*, 2017.

[9] Z. S. Dong, “Social Media Information Sharing for Natural Disaster Response,” *arXiv preprint arXiv:2005.07019*, 2020.

[10] O. J. M. Peña Cáceres *et al.*, “Chatbot System for Data Management: A Case Study of Disaster-related Data,” ResearchGate, 2024.

[11] L. Dupont *et al.*, “Collaborative Virtual Reality Environment in Disaster Medicine: Moving from Single Player to Multiple Learners,” *BMC Medical Education*, vol. 24, no. 422, 2024.

[12] U. Kant, P. Dahiya, R. Priyadarshi, and M. Kumar, “Generative AI in Communication System Simulation and Modeling,” in *Advances in Communication Systems Modeling*, 2025.

[13] P. Dahiya and U. Kant, “Multi-Factor Authentication Methods for Intelligent Systems,” in *Intelligent Systems Security*, CRC Press, 2023.

[14] A. Mahmoud, “Sustainable Virtual Worlds: Implementing Renewable Energy Solutions,” in *Green Metaverse: Fuzzy Logic Approaches to Sustainable Virtual Worlds*, Springer Nature Switzerland, 2026, pp. 111–133.

[15] S. Sharma, “Performance Evaluation of the Deep Learning Based Convolutional Neural Network Approach for the Recognition of Chest X-ray Images,” *Frontiers in Oncology*, vol. 12, p. 932496, 2022.

Appendix

This appendix contains supplementary materials that support the implementation and functionality of the proposed disaster management system. These include code snippets, system flow representations, and additional technical details that are not included in the main body of the report due to their length and complexity.

A.1 Sample Code Snippets

A.1.1. Data Scraping Script (Python)

```
import requests
from bs4 import BeautifulSoup

url = "https://example-news-site.com/disaster-news"
response = requests.get(url)

soup = BeautifulSoup(response.text, 'html.parser')

articles = soup.find_all('article')

data = []

for article in articles:
  title = article.find('h2').text
  link = article.find('a')['href']

  data.append({
    "title": title,
    "link": link
  })

print(data)
```

A.1.2. NLP Preprocessing

```
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
```

```
def preprocess(text):
  words = text.lower().split()
  words = [w for w in words if w not in stopwords.words('english')]

  stemmer = PorterStemmer()
  words = [stemmer.stem(w) for w in words]

  return " ".join(words)
```

A.1.3. Basic Classification Example

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

texts = ["flood warning issued", "earthquake reported", "rescue operation ongoing"]
labels = ["alert", "alert", "rescue"]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(texts)

model = MultinomialNB()
model.fit(X, labels)

prediction = model.predict(vectorizer.transform(["flood alert in city"]))
print(prediction)
```

A.1.4. Object Detection (TensorFlow Lite - Basic Setup)

```
val tflite = Interpreter(loadModelFile())
val inputBuffer = ByteBuffer.allocateDirect(4 * inputSize * inputSize * 3)

tflite.run(inputBuffer, outputBuffer)
```

A.2 System Requirements

A.2.1 Hardware Requirements

- Android smartphone (minimum 4GB RAM recommended)
- Intel i3 / Equivalent
- GPS sensor
- Microphone and speaker

A.2.2 Software Requirements

- Android Studio (latest version)
- Android SDK (API level 26 or above)
- Kotlin programming language
- TensorFlow Lite / ML Kit
- Google Maps API
- Firebase (optional for backend)

A.3 API Configuration Details

The system uses REST APIs to connect all modules.

Key APIs:

- **/fetch-data** → Retrieves processed disaster data
- **/classify** → Sends text to NLP model for classification
- **/chatbot** → Handles user queries and responses
- **/alerts** → Sends real-time notifications to mobile app

Example API Request:

```
POST /classify
{
  "text": "Flood warning issued in Delhi"
}
```

Example API Response:


```
{
 "category": "Alert",
 "confidence": 0.87
}
```


A.4 Sample Use Case Scenarios

Use Case 1: Real-Time Alert

- User opens mobile app
- System fetches latest disaster updates
- User receives notification about nearby disaster

Use Case 2: Chatbot Assistance

- User asks: “What should I do during an earthquake?”
- Chatbot provides step-by-step safety instructions

Use Case 3: VR Training

- User selects “Flood Simulation”
- VR module simulates flood environment
- User learns evacuation procedures

A.5 Limitations and Constraints

- Dependence on availability and reliability of web data sources
- Classification accuracy may decrease with noisy or ambiguous data
- VR module requires compatible hardware for full experience
- Internet dependency for real-time updates (partially solved with offline mode)
- API rate limits from external platforms

A.6 Screens (Description)

1. Mobile App Home Screen:

- Displays latest disaster alerts and notifications.

2. Alert Detail Screen:

- Shows detailed information including location, severity, and instructions.

3. Chatbot Interface:

- Interactive UI for user queries and guidance.

4. Map View Screen:

- Displays disaster locations using Google Maps integration.

5. VR Simulation Screen:

- Allows users to select and experience disaster scenarios.

A.7 Future Enhancement Notes

- Integration with government disaster management systems
- AI-based fake news detection using deep learning
- Multi-language chatbot support
- Real-time crowd-sourced reporting system
- Cloud deployment for scalability
- Advanced predictive analytics using LSTM models

A.8 Additional Notes

- The system is designed using a modular architecture, allowing independent scaling of components
- REST APIs are used for communication between modules
- The mobile application supports offline access using local storage
- The VR module is implemented using Unity with C# scripting

Disaster Management Using Real-Time Web Data, Chatbot Assistance, and VR Simulation

Kartik Chaudhary, Avanish Kumar, Ashutosh Mishra, Animesh Kumar Singh, Bhuvnesh Malik, and Umang Kant

kartikchaudharyy99@gmail.com, kavanishhere@gmail.com, ashutoshbins@gmail.com, tronuprisingclg@gmail.com, bhuvneshmaalik@gmail.com, umang.kant@kiet.edu

Department of CSE (AI&ML)
Krishna Institute of Engineering & Technology (KIET), Ghaziabad, Delhi-NCR, Uttar Pradesh, India

Abstract. The need for rapid access to trustworthy information has become essential because disasters are occurring more frequently and their impact has increased. Our disaster management system which uses real-time data and ML technology to improve both preparedness and response operations. The system provides a mobile application which broadcasts alerts enables users to communicate without internet access and delivers information updates. The VR experience uses Unity to create a training platform which simulates various disaster scenarios.

Keywords: Disaster Management, Web Scraping, NLP Classification, SVM, Naive Bayes, Smart India Hackathon 2024 (SIH1687) project, Chatbot, LLM, Flutter, VR Simulation,

Organization of the Paper

1 Introduction

The researchers developed a complete disaster management system which consists of The system uses Python to collect live web data. The system uses natural language processing (NLP) to categorize information through support vector machine (SVM) and Naive Bayes model. The Unity-based system enables users to

experience disaster training through Virtual Reality disaster simulations. The last 20 years have witnessed a sharp increase in disaster frequency and severity which resulted in 7300 major disasters between 2000 and 2019 that killed 123 million people and affected 4 billion people.[1] The commonality between disaster events shows that scientific and system advancements have improved prediction and preparation capabilities yet people enter critical situations without trustworthy information.[8] The spread of falsehoods together with contradictory communication and disaster knowledge gaps is increasing dangers in areas where people live together and lack basic necessities such as resources in India.[9]

Modern disaster work has to do more than just provide isolated data or reactive alerts. It needs to work in real time, send alerts in different ways, and learn from experience to help whole communities with all parts of the disaster cycle.[6]

There are many technical challenges to making this happen: 1. information from disasters comes from many different sites and formats; 2. we need to sort fact from fiction, find who to trust; 3. we need to send early alerts in seconds or minutes, not hours or days, to save lives and understand what is happening[9] [7] . To meet these needs, I have built a disaster system that joins together: Python-based real-time web scraping, NLP classifiers based on SVM and Naive Bayes, a smart chatbot that mixes rules with large language models for better user talks, a mobile app built with Flutter that works without a net but alerts you based on your location, and Unity VR simulations that teach about disaster events by letting users experience them as if they were in the disaster cycle. This multi-layer plan will help communities be ready, make information better, and get more lives saved.

2 Problem Statement and Scope

Even though the world has come so far in how we can forcast these problems and how we can respond to them communities are still faced with a lot of problems trying to get the right and accurate information to act on in time as its happening. The lack of a centralized platform to consolidate, verify and distribute accurate and verified information about disasters is a critical flaw that can be exploited.. Another major concern is the lack of real time automated verification of disaster related content, social media and other streams contain useful information but can also be rife with rumours or false alarms that just create confusion and panic or worse[8]. There needs to be machine learning based filtering and categorization in order to distinguish between high priority alerts and low priority or irrelevant information[9] [7].

Also disaster management tools rarely feature interactive or immersive education tools, which would help ensure the public is prepared for disasters and are aware of the safe-response procedures that are necessary during emergencies. [6] [3].

The scope of this work is bounded by the goal of developing a comprehensive, multi-modal platform that improves disaster preparedness and real-time

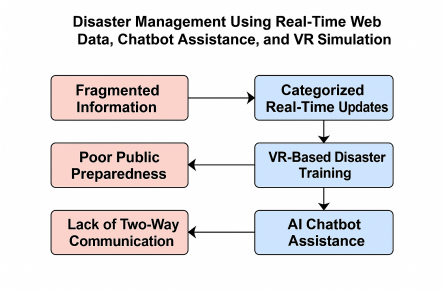


Fig. 1. Illustration of challenges in current disaster information systems.

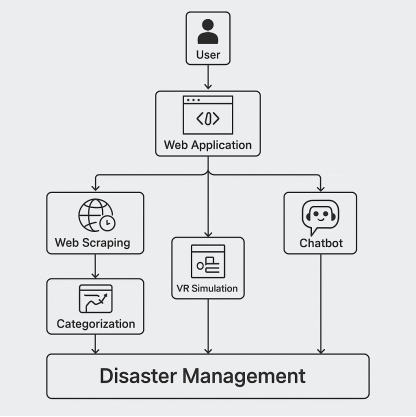


Fig. 2. System Workflow Representation of the Proposed Model

response. The proposed system focuses on: (1) automated acquisition and classification of disaster-related information, (2) accessible delivery of alerts and safety resources through a mobile application with offline capability, (3) an AI-assisted chatbot for interactive guidance, and (4) a VR module for awareness and training. the system does not do satellite based forecasting or large sensor networks for live rescue coordination it does not replace or tie into the real world it aims to support our existing institutions to make our communities more aware prepared and reliable.

2.1 Threat and Challenge Landscape

In addition to the poor communication, many tech and process threats can limit the use of modern disaster-management systems. One threat is fake news and fake alerts on social media[9]. When there is a disaster, posts that are fake can spread faster than the news reports that are true. It becomes more important to filter, classify, and check the reports as soon as possible. This in turn, calls for robust NLP models that are capable of handling poorly structured, differently spoken, and context-related content efficiently. [7]

Lag in data pipelines is posed as another threat. The situation in which it takes a considerable amount of time for data to be scraped, worked on, or sent out is definitely a problem, especially for those cases that are rapidly evolving, such as floods or earthquakes. In order to get rid of this problem, it is necessary to have an excellent design and small models that can work at a high speed.

Also, watch out for platform problems. Some social platforms, like X, are limiting how much you can scrape or view. [5] These limits show why offline options, using less data, and very small models are important if you want everyone to use new tech without issues.[6]

2.2 Ethical Considerations

Definitely if a system with AI that is involved in emergencies will be used, then the data safety should be priortized first. Posts, GPS and the way people use a site can all be sources of private information. The system has to secure all that also anonymize user IDs, and comply with data regulations in order to prevent hacking or misuse.[13] Some updates may be overlooked. To solve the problem, the system should be vigilant about bias and it should re-train itself with data from everywhere so that things remain fair for everyone.[8] Getting the balance right between where alerts come from and where they go is key, because people need to get alerts from trusted sources that are accurate and on time.[4]

3 Review of Literature

Intelligent Disaster Response is becoming more aware as we find more efficient ways to handle crisis information. New tools are using information, data and technology to help people share information and figure out what to do in case

of a disaster.[2]. Since online information is rapidly changing and can be quite disorganized it is hard to decide what is true confirm details and give updates quickly.[7]

A lot of studies has shown that social media like Twitter and Facebook can share quick updates during disasters. Techniques using SVM, Naive Bayes and Transformer models are good at sorting short texts, but fake stuff, different languages, and suspect sources create problems. [8]. Research on disaster chatbots says they can help send info fast, list first-aid steps, and chat back and forth during a crisis. But many systems only use set answers and don't work well when disaster situations change quickly.[9]

At present, language models provide versatile assistance and have a better understanding of the situation. Besides that, Virtual Reality is progressively being employed for teaching and training in the occurrence of the disaster. VR assists people in concentrating, recalling things, and getting prepared by presenting them with the dangerous situations of floods, earthquakes, and fires.[4] [10]

All in all, many studies show one key problem: though searching social media, designing chatbots, and training with VR can be made better, these systems often are separate. No full disaster tool that takes info from live web data, sorts it automatically, talks back and forth, and gives users VR training in one simple tool. This study makes new disaster tool that mix these tools in one easy-to-use system, using NLP tools, AI help, phone alerts, and VR training. [6] [11]

4 Our Approach

The system is designed to look simple with blocky layers. It is meant to get, sort, share, and show disaster data fast. There are five main parts in the system: a Python script engine to get data, a machine learning step to sort text, a Node.js server to manage programs, a Flutter app for phones to show updates fast, and a Unity VR tool for disaster practice. Each part can work at its own but works well with REST APIs, making it is easy to be updated.[5]

4.1 System Architecture Overview

The system uses a modular service based design with five main parts, those are - a Node.js API server, small Python app for NLP driven text sorting, MongoDB database for data storage, a Flutter app for real time alerts and offline use, and at last a Unity powered VR disaster training tool. These parts connect via small REST APIs to let them run individually, scale easily, and share data quickly. The different sub chapters describe the parts of the system in detail:

- **4.2 Data Acquisition Pipeline** – Python scripts to automate data gathering, cleaning, and ranking of newly occurred disasters.
- **4.3 NLP Classification Engine** – SvM, Naive Bayes classifiers that identify the types of alerts and recognize the presence of false data.

– **4.4 AI Chatbot Architecture** – combination of rules and LLM for flexible dialog.
– **4.5 Flutter Mobile Application** – functioning locally, giving you alerts dependent on your location, and enabling you to communicate with others.
– **4.6 VR Simulation Module** – Unity to construct the 3D environment for disaster drills.

4.2 Data Acquisition Pipeline

The data collection system is getting the news about disasters that are happening from social media and the news that is already known by using a Python tool. Static web pages are read with Requests, while JavaScript-loaded pages are handled by Selenium. The system is operating on a cron schedule which is regularly updating the data like text, time, images, and links to the original site.

In order to focus on the most valuable information, a relevance score R is given to each scraped item which is determined by the keyword intensity (K), time relevance (T), and source credibility (S):

$$R = \alpha K + \beta T + \gamma S$$

where α, β, and γ are empirically tuned weights. Posts with a score above a set limit get sent to the NLP classification system (Section 4.3). This small process makes sure that the updates are received on time and only the important information is gathered.

4.3 NLP-Based Classification Engine

The classifier tool gets each post you pull out and does a common NLP step: split it up into words, make all lower case, remove common words with little meaning, and cut words down to their root form.

$$x = f(\text{tokenize, lower, remove stopwords, stem})$$

The text vector is then run through TFIDF. It then goes to two trained models: an SVM classifier that can sort into more than one group and a Multinomial Naive Bayes model made for short, light text. Its predicted category $\hat{y}$ is computed as:

$$\hat{y} = \arg\max_{c \in C} P(c \mid x)$$

where C includes five disaster-related classes: *Alerts*, *Government Notices*, *Rescue Updates*, *Public Advisories*, and *Casualty Reports*. Using SVM and Naive Bayes together gives you a good mix of correct answers and quick results so you can tell the different big parts of fresh data as it rolls in.[7][15]

4.4 AI Chatbot Architecture

The chatbot system has a mixed design of rule-based intent classifier and large language model fallback used for difficult or ambiguous questions.[4] [12] When a user asks q, the system checks for matching keywords or predefined intent templates; if it cannot identify the intent with high confidence, it sends q to a lightweight API powered by a language model that generates responses on the fly. The intent determination involves choosing from:

$$I = \arg\max P(I_k \mid q)$$

where I_k is the most likely class of intent. flow of conversation is kept with a very simple FSM, able to keep multi-turn dialogs going for all the pre disaster, during disaster and post disaster circumstances. this is very useful for simple queries, but it is also more flexible for anything else the user wants to do.

4.5 Flutter Mobile Application

The app was made with Flutter and is the main screen for learning about disaster updates. It is equipped with a cache system, therefore it can function offline and utilizes SQLite, so that you can access key information even if you don't have a signal. Real-time alerts are delivered through FCM. Using the Google Maps API, you can receive warnings about geolocation or look at the hazards. With Bloc/Riverpod, the app's behavior is made to be predictable and simple to change in the user interface.

the REST calls to the Node.js backend are minimal and swift, also a low-bandwidth mode is included so that the app remains responsive and is more convenient to use with limited internet connection.

4.6 VR Simulation Module

The virtual reality segment employs the Unity engine to create an immersive experience for disaster training.[14] Unity's SceneLoader, which is the means of changing the user interface from one to another, is used by users to navigate through different disasters such as floods, quakes, and fires in brief sessions. Earthquake scenes use Rigidbody physics to shake the ground, flood scenes use easy NavierStokes math to make water look real. Custom code in C# makes the user play their part and decide what to do next in our Flutter app.[6]

In this way we can send data both ways and then people can sign in and pick scenes from both.

5 Results and System Evaluation

Evaluate the system across its key parts to see how well it classifies, how fast it is, how quick it responds, and how good it is for users. Run tests on a set of posts

taken from news sites and social media sites over two weeks[7]. The NLP classifier worked well most of the time, with the SVM method being more accurate in classifying many types of classes, while the Multinomial Naive Bayes method was faster when working with short text inputs. The scraping tool was very fast even with it running all the time on a schedule, and the mobile app worked reliably whether connected or not. The system results are written down in Table 1. This test shows that the full system can take and send disaster updates nearly

Table 1. System Performance Evaluation

Component	**Performance Metric**
Scraping Pipeline	1.2 s avg. extraction latency
SVM Classifier	82% accuracy
Naive Bayes Classifier	76% accuracy (faster inference)
Chatbot Response Time	0.5 s avg.
Flutter App Offline Load	190 ms
VR Module Frame Rate	55–60 FPS

as soon as they happen. The conversation-agent in hybrid-mode was efficient in handling both types of questions i.e., filled as well as empty, and the Unity VR component was running smoothly without any lag and was nice for training purposes. In sum, this experiment demonstrates the innovative system as a vehicle to propagate information in a swift manner, sustain the engagement level of the users, and foster the development of a robust preparedness strategy.[11]

6 Case Studies of Past Disasters

Let's look at three disasters to see how this system could help. Each disaster had trouble with communication, keeping people informed, and getting useful info out fast.

Kerala Floods (2018) The 2018 Kerala floods displaced more than a million people and the communication of dam releases, rescue points, and shelters was very problematic. In the flooded situation, the use of social media was very high; however, there were many false and mixed updates. With the new technology, people should have been able to see everything that was happening live; also, the system may have automatically differentiated the news by using computer vision in order to eliminate false news, and in that way, targeted notifications could have been sent to the correct people. Moreover, discussing through VR about how to behave during floods might have helped people grasp the situation as well as the evacuation process.

Nepal Earthquake (2015) That huge 7.8 quake cut off Nepal leaving tons without any word on what to do after or where to find help. Now if they had the

app, it would have been saved on their phones and worked even with cruddy or no internet. Folks could still get safety tips. The bot could've been a lifesaver, showing first responders and locals first aid, who to call and custom tips for their area.[8].

Cyclone Amphan (2020) Cyclone Amphan caused a lot of harm in India and Bangladesh areas that were largely rural and where people did not have enough time to listen to a warning and leave. The bot's data pipeline might have combined the warnings from weather bureaus and town halls into one clear and simple notification. The interactive map could have shown people where it would have been safe to go, and the VR game could have prepared them for the future by teaching them what to do when a cyclone was approaching.[9]
These stories how how a system, giving information right away and is easy to use, could really help keep people out of danger, squash fake news, and be a big help when disaster strike.

Figure 4 compares how often disasters happened and how many people died between 1900 and 2020.

7 Future Scope

The system design is adaptable and can be developed in numerous ways to make it more efficient and practical in everyday life. The main points of the future roadmap are:

- **Integration with NDMA and Government APIs:** Never rely on intermediary services for alerts, safety procedures, and threat maps, take them directly from the source for more security.
- **Predictive Modeling:** Use LSTM as a supplementary tool for weather prediction by forecasted flood height, storm power, or aftershock risk .[2]
- **Multilingual Support:** Incorporating chatbot and mobile words in other languages would facilitate communication with people in remote areas where languages differ or in villages. [2]
- **Crowdsourced Reporting:** User-generated hazard reports with trust scores could also be a way to prompt local first responders to arrive on time and create a rapid response scene account.
- **Scalable Cloud Deployment:** Moving to smaller servers on AWS or GCP means apps can grow without restarting.

These changes will help make the system faster, stronger, and better at handling disasters.

8 Conclusion

This paper introduces a disaster plan equipped with all the latest tools you might need. It is capable of receiving data in real-time, categorizes it using

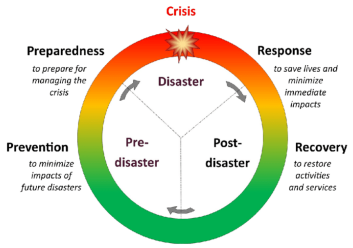


Fig. 3. Disaster Management Cycle: the phases of mitigation, preparedness, response and recovery

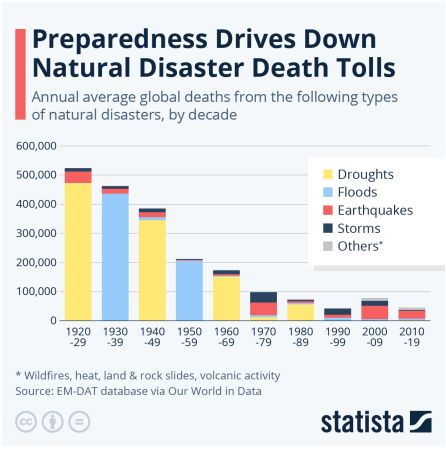


Fig. 4. Global Disaster Trends: Number of Events and Deaths b/w 1900–2020

an NLP classifier, employs a talking bot that combines rules and a LLM, and operates a VR training program that gives you the sensation of being there. This plan addresses the major issues of the modern disaster world - the data mess, fake news, and rapid spread of false information - by using a configuration that can add new features and become big. Testing results are a testament to its performance and efficiency for tech people with the SVM scoring 82, the Naive Bayes being quick, the scraping tool being low delay, and the Flutter app being very fast and able to run even if offline or on a slow network. The hybrid chatbot users the system better by giving safe and correct answers from rules or flexible answers from a LLM, and the Unity VR feature offers high quality, smooth, and real-feeling training.[6] [4]

This project is not just about the technology it shows why saving lives is important. Disasters don't have a "coming soon" period, they happen so suddenly. We have to prepare ourselves in advance for them. Our design also means it can adapt to new data, is not restricted to language and can also help government rescue agencies.

References

1. K. Sapountzaki, "Risk Mitigation, Vulnerability Management, and Resilience under Disasters," *Sustainability*, vol. 14, no. 6, pp. 1–8, 2022. [Online]. Available: https://doi.org/10.3390/su14063589
2. H. L. Tay, R. Banomyong, P. Varadejsatitwong, and P. Julagasigorn, "Mitigating Risks in the Disaster Management Cycle," *Advances in Civil Engineering*, vol. 2022, Article ID 7454760, 2022. [Online]. Available: https://doi.org/10.1155/2022/7454760
3. Anonymous, "Poverty and Disaster Resilience," *Independent Research Paper*, 2020.
4. A. Rossi, "The ERMES Chatbot: A Conversational Communication Tool for Improved Emergency Management and Disaster Risk Reduction," *International Journal of Disaster Risk Reduction*, vol. 112, 2024. [Online]. Available: https://doi.org/10.1016/j.ijdrr.2024.104792
5. K. Kopanov, V. Varbanov, and T. Atanasova, "Leveraging Social Media Data and Artificial Intelligence for Improving Earthquake Response Efforts," *arXiv preprint arXiv:2501.14767*, 2024. [Online]. Available: https://arxiv.org/abs/2501.14767
6. S. Khanal et al., "Virtual and Augmented Reality in Disaster Management Technology: A Literature Review of the Past 11 Years," *Frontiers in Virtual Reality*, vol. 3, 2022. [Online]. Available: https://www.frontiersin.org/articles/10.3389/frvir.2022.843195/full
7. F. Alam, F. Ofli, and M. Imran, "CrisisMMD: Multimodal Twitter Datasets from Natural Disasters," *arXiv preprint arXiv:1805.00713*, 2018. [Online]. Available: https://arxiv.org/abs/1805.00713
8. T. H. Nazer et al., "Intelligent Disaster Response via Social Media Analysis: A Survey," *arXiv preprint arXiv:1709.02426*, 2017. [Online]. Available: https://arxiv.org/abs/1709.02426
9. Z. S. Dong, "Social Media Information Sharing for Natural Disaster Response," *arXiv preprint arXiv:2005.07019*, 2020. [Online]. Available: https://arxiv.org/abs/2005.07019
10. O. J. M. Peña Cáceres et al., "Chatbot System for Data Management: A Case Study of Disaster-related Data," *ResearchGate*, 2024. [Online]. Available: https://www.researchgate.net/publication/334127275
11. L. Dupont et al., "Collaborative Virtual Reality Environment in Disaster Medicine: Moving from Single Player to Multiple Learners," *BMC Medical Education*, vol. 24, no. 422, 2024. [Online]. Available: https://doi.org/10.1186/s12909-024-05429-8
12. U. Kant, P. Dahiya, R. Priyadarshi, and M. Kumar, "Generative AI in Communication System Simulation and Modeling," in *Advances in Communication Systems Modeling*, 2025. [Online]. Available: https://doi.org/10.2174/9789815324693125040l
13. P. Dahiya and U. Kant, "Multi-Factor Authentication Methods for Intelligent Systems," in *Intelligent Systems Security*, CRC Press, 2023. [Online]. Available: https://doi.org/10.1201/9781032630748-7
14. A. Mahmoud, "Sustainable Virtual Worlds: Implementing Renewable Energy Solutions," in *Green Metaverse: Fuzzy Logic Approaches to Sustainable Virtual Worlds*. Cham: Springer Nature Switzerland, 2026, pp. 111–133.
15. S. Sharma, "Performance Evaluation of the Deep Learning Based Convolutional Neural Network Approach for the Recognition of Chest X-ray Images," *Frontiers in Oncology*, vol. 12, p. 932496, 2022.

KIET UNIVERSITY
DELHI-NCR, INDIA
(Under Section 3 of the UGC Act, 1956)

KARTIK KIET <kartik.2226cseml1076@kiet.edu>

ICCDM 2026: Paper Notification for Paper ID - ICCDM2026-V18

1 message

ICCDM Conference <iccdm.conf@gmail.com> Tue, Feb 17, 2026 at 3:01 PM
To: "umang.kant@kiet.edu" <umang.kant@kiet.edu>, "avanish.2226cseml104@kiet.edu" <avanish.2226cseml104@kiet.edu>, animesh.2226cseml13@kiet.edu, "ashutosh.2226cseml1@kiet.edu" <ashutosh.2226cseml1@kiet.edu>, "kartik.2226cseml1076@kiet.edu" <kartik.2226cseml1076@kiet.edu>, "bhuvnesh.malik@kiet.edu" <bhuvnesh.malik@kiet.edu>
Cc: "Dr. Vikash Yadav" <vikas.yadav.cs@gmail.com>

International Conference on Cyber Security, Data Science & Machine Learning (ICCDM-2026)

Dear Author(s),

Greetings from - ICCDM 2026!

We congratulate you that your paper with submission ID **'ICCDM2026-V18'** and Paper Title **' Disaster Management Using Real-Time Web Data, Chatbot Assistance, and VR Simulation'** has been accepted for publication in the **Proceedings of ICCDM 2026- Springer LNNS Series** [Indexing: zbMATH, Scopus, DBLP, EI Compendex & SCImago - Approved]. This acceptance means your paper is among the top 15% of the papers received/reviewed.

To secure your participation in this prestigious event, we request that you complete your early-bird registration by 28th February 2026.

You are requested to do the registration as soon as possible and submit the following documents to iccdm.conf@gmail.com at the earliest.

1. Final Camera-Ready Copy (CRC) as per the springer format- (See https://www.iccdm-conf.com/downloads)
2. Copy of e-receipt of registration fees. (For Registration, see https://www.iccdm-conf.com/registration)
3. The final revised copy of your paper should also be uploaded via Microsoft CMT.
4. Do not create another ID, just upload the revised version on the existing Microsoft CMT ID only

It is suggested to cite the relevant latest papers matching the area of your current research paper.

The paper prior to submission should be checked for plagiarism from licensed plagiarism softwares like Turnitin/iAuthenticate etc. The similarity content should not exceed 15%.

Pay registration fees via online portal:

Submission of Final CRC and Registration Documents – ICCDM2026-V18

3 messages

Kartik Chaudhary <kartikchaudharyy99@gmail.com> Sat, 28 Feb, 2026 at 11:32 am
To: iccdm.conf@gmail.com

Dear Sir/Madam,

Greetings from our side.

With reference to the acceptance notification for our paper titled **"Disaster Management Using Real-Time Web Data, Chatbot Assistance, and VR Simulation"** (Paper ID: ICCDM2026-V18), we are pleased to inform you that we have completed the early-bird registration process.
As requested, we have attached the documents.
With Regards,
Kartik Chaudhary
(On behalf of all co-authors)
Secondary email: kartik.2226csem1076@kiet.edu

5 attachments

- **Springer Disaster Management.pdf**
  3.4 MB
- **Springer Disaster Management.docx**
  568 KB
- **yono_receipt_605911797578_20260228111843.pdf**
  70 KB
- **Plag Report Springer__Disaster_Management (1).pdf**
  3.5 MB
- **AI Report Springer__Disaster_Management .pdf**
  3.5 MB

ICCDM Conference <iccdm.conf@gmail.com> Sat, 28 Feb, 2026 at 5:38 pm
To: Kartik Chaudhary <kartikchaudharyy99@gmail.com>

Dear Author,

Thanks for the registration. We will be sending further details soon. Also we will be seeing the final file that you have sent as per the suggestion in the acceptance mail.
[Quoted text hidden]

ICCDM Conference <iccdm.conf@gmail.com> Mon, 2 Mar, 2026 at 8:56 am
To: Kartik Chaudhary <kartikchaudharyy99@gmail.com>

Dear Author,

Thanks for the registration and paying 11000/- as an registration fees. We will be sending further details soon. Also we will be seeing the final file that you have sent as per the suggestion in the acceptance mail.

[Quoted text hidden]

Springer__Disaster_Management.pdf

Delhi Technological University

Document Details

Submission ID
trn:oid:::27535:129454927

Submission Date
Feb 28, 2026, 10:39 AM GMT+5:30

Download Date
Feb 28, 2026, 10:41 AM GMT+5:30

File Name
Springer__Disaster_Management.pdf

File Size
3.2 MB

11 Pages

3,935 Words

20,418 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

1% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Cited Text
- Small Matches (less than 8 words)

Match Groups

5 Not Cited or Quoted 1%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
0% Publications
1% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.